Lab 05 – Queue

Task 01:

Emily is a manager at a small logistics company that specializes in delivering packages to local businesses. To streamline the package handling process, Emily decides to develop a C++ program using a QUEUE data structure with a maximum capacity of 6 packages.

The program will include three key functions:

1. enQueue() to add packages to the delivery queue,
2. deQueue() to remove packages once they are delivered, and
3. Display() to show the current status of the delivery queue.

Emily plans to test the program with a series of procedures based on the company's typical operations

Code:

```
#include<iostream>
using namespace std;

const int n = 6;
int packages[n];
int f = -1, r = -1;

void enQueue(int val){
    if(r == n-1){
        cout << "Queue is full!\n";
        return;
    }

    if(f == -1){
        f = 0, r++;
        packages[r] = val;
        return;
    }

    r++;
    packages[r] = val;
    return;
}

void deQueue(){
    if(f == -1){
        cout << "Queue is empty!\n";
        return;
    }
```

```
    if(f == r+1){
        cout << "Once upon a time queue was full, but right now it's empty!\n";
        return;
    }

    cout << "Deleting: " << packages[f] << "\n";
    f++;
    return;
}

void display(){
    cout << "Current Queue: ";
    for(int i = f; i <= r; i++) cout << packages[i] << " ";
    cout << "\n";
    return;
}

int main(){

    enQueue(10);
    enQueue(7);
    enQueue(4);
    enQueue(8);
    enQueue(2);
    enQueue(15);

    display();

    enQueue(25);

    deQueue();

    display();

}
```

Output:

```
Current Queue: 10 7 4 8 2 15
Queue is full!
Deleting: 10
Current Queue: 7 4 8 2 15
```

Task 02:

Alice, the manager of a popular movie theater, is seeking to streamline the ticket booking process for her customers. To achieve this, she plans to develop a C++ program using a Circular QUEUE data structure with a maximum capacity of 6 tickets.

The program will include three key functions:

1. enQueue() to add tickets to the booking system,
2. deQueue() to remove tickets once they are booked,
3. and Display() to show the current availability of tickets.

Alice intends to test the functionality of her program with a series of procedures reflectingtypical movie theater operations

Code:

```
#include<iostream>
using namespace std;

struct Movie{
    string name;
    int price;
};

const int n = 6;
Movie movies[n];
int f = -1, r = -1;
int currSize = 0;

void enQueue(Movie m){
    if(currSize == n){
        cout << "No more movies can added.\n";
        return;
    }

    if(f == -1 && r == -1){
        f = r = 0;
        movies[r] = m;
        currSize++;
        return;
    }

    r = (r+1)%n;
    movies[r] = m;
    currSize++;
}
void dequeue() {
```

```
    if(currSize == 0){
        cout << "There are no movies in queue.\n";
        return;
    }

    f = (f+1)%n;
    currSize--;
}

void display(){
    if(currSize == 0){
        cout << "No movies in queue to display.\n";
        return;
    }

    int i = f;
    cout << "\n\t*Movies in Queue*\n";
    for(int count = 0; count < currSize; count++){
        cout << movies[i].name << ", " << movies[i].price << "\n";
        i = (i+1)%n;
    }
}

int main(){

    enQueue({"Avatar", 10});
    enQueue({"Inception", 7});
    enQueue({"The Dark Night", 4});
    enQueue({"Intersettler", 8});
    enQueue({"The Matrix", 2});
    enQueue({"The Star Wars", 15});

    cout << "\n";
    display();
    cout << "\n";

    enQueue({"Divergent", 25});
    cout << "\n";
    dequeue();
    dequeue();
    dequeue();
    dequeue();
    dequeue();
    dequeue();
    display();
}
```

Output:

```
        *Movies in Queue*
Avatar, 10
Inception, 7
The Dark Night, 4
Intersettler, 8
The Matrix, 2
The Star Wars, 15

No more movies can added.

No movies in queue to display.
```

Task 03:

FreshMart Grocery Store wants to implement a checkout system using a **Circular Queue** data structure. The system should manage a maximum of **10 customers** at a time.

Customers join the queue after finishing shopping, and they are removed once their checkout process (scanning, payment, receipt) is completed.

The program must:

- Add customers to the queue.
- Remove customers after checkout.
- Maintain a maximum capacity of 10 customers.
- Display an error message if the queue is full or empty.

Code:

```
#include<iostream>
using namespace std;

const int n = 10;
int customers[n];
int f = -1, r = -1;
int currSize = 0;

void enqueue(int customerNumber){
    if(currSize == n){
        cout << "No more customers can be added.\n";
        return;
    }

    if(f == -1 & r == -1){
        f = r = 0;
        customers[r] = customerNumber;
```

```
        currSize++;
        return;
    }

    r = (r+1)%n;
    customers[r] = customerNumber;
    currSize++;
}

void dequeue(){
    if(currSize == 0){
        cout << "There are no customers in queue.\n";
        return;
    }

    f = (f+1)%n;
    currSize--;
}

void display(){
    if(currSize == 0){
        cout << "No customers in queue to display.\n";
        return;
    }

    int i = f;
    cout << "\n\t*Customers in Queue*\n";
    for(int count = 0; count < currSize; count++){
        cout << customers[i] << " ";
        i = (i+1)%n;
    }
    cout << "\n";
}

int main(){

    enqueue(1);
    enqueue(2);
    enqueue(3);
    enqueue(4);
    enqueue(5);
    enqueue(6);
    enqueue(7);
    enqueue(8);
    enqueue(9);
    enqueue(10);
```

```
    cout << "\n";
    display();

    cout << "\n";
    enqueue(11);

    dequeue();
    dequeue();
    dequeue();
    cout << "\n";
    display();
}
```

Output:

```
        *Customers in Queue*
1 2 3 4 5 6 7 8 9 10

No more customers can be added.


        *Customers in Queue*
4 5 6 7 8 9 10
```